계층적 강화학습에서의 표준적 계층 추가 방안:
Timely Hierarchical Elaborated FeUdal Networks

Standard Hierarchy addition method in Hierarchical Reinforcement Learning:
Timely Hierarchical Elaborated FeUdal Networks

김종원
단국대학교
kjw65144128@gmail.com

요 약

계층적 강화학습은 두개 이상의 계층을 이용하는 강화학습 방법으로 보상이 희소하고 지연된 환경에서 기존 강화학습보다 높은 성능을 보인다. 셋 이상의 계층 사용이 학습을 더욱 빠르게 한다는 점이 밝혀졌음에도 네트워크 구조 및 학습 방법에 대한 논의가 제대로 이루어지지 못하고 있다. 본 논문은 두개의 계층을 이용하는 모델에 계층 하나를 추가한다. 그럼에도 안정적으로 학습할 수 있는 네트워크 구조와 학습 방법을 제안하며 Four Rooms 환경에서 그 실효성을 검토한다. 과거의 연구에서 밝혀진 바와 같이, 계층이 추가된 모델이 더욱 빠른 학습 속도를 보였다. 제안하는 모델 THE FUN 은 둘 이하의 계층을 사용하는 모델에 비해 빠르게 학습했다. 그럼에도 더욱 안정적으로 학습할 수 있음을 확인할 수 있었다.

1. Introduction

전통적인 강화학습에 계층을 더해 복잡하고 어려운 문제를 해결하려는 시도는 큰 성공을 거두었다. 여러 계층이 수직적으로 연결되어 있으며, 각 계층은 보상을 최대화하기 위해 서로 다른 역할을 수행한다. 풀어내기 어려운 문제라도 여러 계층이 서로 다른 기능을 수행하며 나누어서 다루어 주면 풀어낼 수 있다. 이렇게 계층적 강화학습 (hierarchical reinforcement learning; HRL)은 전통적 강화학습이 풀어내지 못하는 문제를 풀어낸다. 특히 보상이 희소하고 지연된 환경에서 기존 강화학습 모델에 비해 좋은 성능을 보여준다. [1]

최근 이러한 HRL 에서 셋 이상의 계층 사용은 학습을 가속화한다는 것이 밝혀졌으나[2] 네트워크 구조 및 학습 방법의 구성에 대해서는 제대로 논의되지 않았다. Vezhnevets 는 Manager-Worker 구조에서 Manager 가 생성한 goal 에 대해 worker 가 intrinsic motivation 을 받는 형태의 HRL 모델의 성능이 뛰어남을 보였다.[3] 여기서 우리는 계층을 어떤 방식으로 쌓고 학습하는지에 따라 성능이 다름을 보이고자 한다.

계층을 추가하는 것은 탐험 상에 이점을 제공함으로 학습을 빠르게 하며, 다계층이 존재하는 HRL 은 상위 계층이 계산하는 장기적 정보를 하위 계층의 목적으로 사용하고 내재적 보상으로 목적을 따르도록 유도함으로 더욱 의미 있는 탐험이 가능하다. 하지만 계층의 증가는 모델 복잡성의 증가를 수반한다. 이 논문에서는 계층을 추가하면서도, 성능을 높이고 학습의 안정성을 높일 수 있는 방법을 제안한다.

2. Model Architecture

2.1 Task

19X19 크기의 Four Rooms 환경을 사용한다. Four Rooms 환경은 벽으로 구분되어 있는 4 개의 방으로 이루어진 환경이다. 각 방을 이어주는 통로를 통해 방을 옮겨 다니며 목적지를 찾아야 한다. 취할 수 있는 행동으로는 forward 와 right, left 가 있다. right 과 left 로 에이전트의 이동 방향을 변경하고 forward 로 이동할 수 있다.

2.2 Model

본 연구에서 우리는 Manager-Worker 구조를 n+1 개의 hierarchy 구조로 확장하였다.

n=0 일 때, 모델은 k=0 에이전트 하나로 구성되어 있다. k=0 에이전트는 A3C 모델[4]이다. A3C 는 하나의 전역 네트워크와 여러 개의 에이전트를 사용하는 방법이다. 에이전트들이 환경과 상호 작용하며 모은 샘플을 통해 전역 네트워크가 학습한다. 전역 네트워크의 학습은 Actor-Critic 방식으로 진행한다. Critic 이 각 상태의 가치를 예측한다. 예측한 값으로 advantage, $A_t = \left(R_t - V_t(x_t;\theta)\right)$ 를 계산해 취한 행동을 평가하는 값을 만들 D수 있다.

Actor 는 $\nabla\pi_t = A_t\nabla_\theta log\pi(a_t|x_t;\theta)$로 advantage 를 이용해 정책을 만든다.

n=1 일 때는 FUN 모델과 동일하게 구성되었다. FUN 은 워커모듈, 매니저 모듈 총 두개의 계층으로 이루어진 계층적 강화학습 모델이다. 매니저 모듈은 목표를 생성하여 워커 모듈에 전달하며, 이 과정에서 매니저 모듈의 목표를 잘 따르면 추가적인 보상을 받도록 구성되어 있다. 추가적인 보상은$r_t^I = 1/c_s\sum_{i=1}^{c} d_{cos}(s_t - s_{t-i}, g_{t-i})$로 계산되어 워커 모듈이 받는 보상을 $r_t + \alpha r_t^I$ 로 구성되며 A_t^D 는 advantage 로 $\left(R_t + \alpha R_t^I - V_t^D(x_t;\theta)\right)$로 구성된다. 이 모델은 Long-term credit assignment, memorisation 과 관련한 환경에서 뛰어난 성능을 보인다. FUN 모델은 k=0 에이전트에 더해 k=1 에이전트가 존재하는 구조로 구성되어 있다. k=0 에이전트는 FUN 의 워커 모듈, k=1 에이전트는 매니저 모듈과 같다.

n=2 는 우리가 제안하는 The FUN 모델로, k=2 에이전트는 입력 받은 z_t 를 k=2 에이전트 내부의 상태 표현 $s_t^{(2)}$ 로 변환하고 k=1 에이전트에게 전달할 목표 $g_t^{(2)}$를 반환한다. 또한 k=1 에이전트는 $g_t^{(2)}$ 를 받아 $g_t^{(2)}$ 로 변환해 k=1 에이전트에 전달한다. k=0 에이전트는 이를 의사 결정에 이용한다. 각 k=0, k=1, k=2 에이전트를 Hierarchy(0)과 Hierarchy(1), Hierarchy(2)로 표현하겠다.

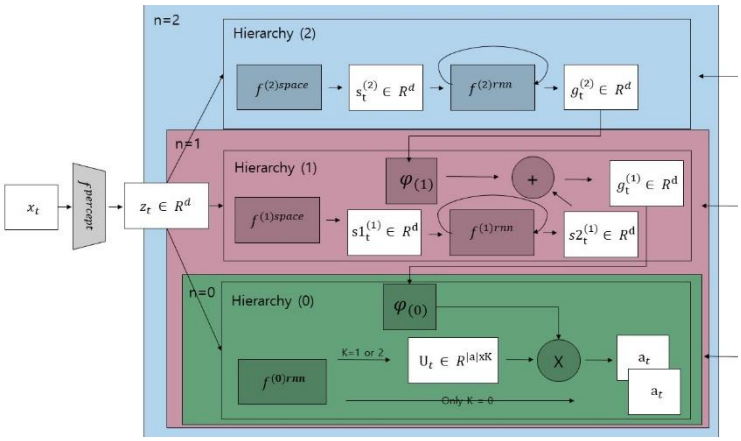


Figure 1. The FUN 설계도

각각의 연산 표현은 다음과 같이 표현할 수 있다.

$$z_t = f^{percept}(s_t)$$
$$s_t^{(2)} = f^{(2)space}(z_t); h_t^{(2)}, \widehat{g_t^{(2)}} = f^{(2)rnn}\left(s^{(2)}\mathrm{t}, \mathrm{h}^{(2)}t-1\right);$$
$$g_t^{(2)} = \widehat{g_t^{(2)}} / \left\|\widehat{g_t^{(2)}}\right\|$$
$$s1_t^{(1)} = f^{(1)space}(z_t); h_t^{(1)}, s2_t^{(1)} = f^{(1)rnn}\left(s1^{(1)}\mathrm{t}, \mathrm{h}^{(1)}t-1\right)$$
$$\widehat{g_t^{(1)}} = s2^{(1)}\mathrm{t} + \phi(1)\left(g_t^{(2)}\right); g_t^{(1)} = \widehat{g_t^{(1)}} / \left\|\widehat{g_t^{(1)}}\right\|$$
$$h_t^{(0)}, U_t = f^{(0)rnn}\mathrm{t}\left(\mathrm{z_t}, \mathrm{h}^{(0)}t-1\right); \pi_t = softmax\left(U_t\phi\left(g_t^{(1)}\right)\right)$$

이 외에도 또 하나의 The FUN 모델을 만들었다. Hierarchy(1)에 전달된 $g_t^{(2)}$에 softmax 를 취해 $s2_t^{(1)}$와 더하는 모델이다. 이를 The Fun softmax 라 하겠다.

2.3. Learning

Hierarchy(0)이 1 의 업데이트 주기를 가지도록 하여, 매번 행동을 변경해주는 반면, Hierarchy(1)는 10 으로 설정해준다. Hierarchy(2)은 Hierarchy(1)의 주기보다 긴 20 을 업데이트 주기로 가지게 설정함으로 상위 계층이 중간/하위 계층에 비해, 중간 계층이 하위 계층에 비해 더욱 장기적 관점을 지닐 수 있게 한다.

Hierarchy(2), Hierarchy(1), Hierarchy(0) 각각은 입력 z_t를 공유한다. z_t는 현재 상태의 잠재 표현으로, 상태 관측값인 x_t의 변환값이다. Percetpual module 을 통해 계산한다.

The Fun 의 Hierarchy(2)는 자신이 만든 목표를 Hierarchy(1) 연산 과정에 추가하여 개입한다. Hierarchy(1)이 내부적으로 계산한 $s2_t^{(1)}$ 에 $\phi_m\left(g_t^{(2)}\right)$를 더해주어 영향력을 행사한다. 계층이 높아 장기적인 정보를 얻을 수 있는 Hierarchy(2)가 자신이 만든 목표를 통해 Hierarchy(1)도 장기적 정보를 고려할 수 있도록 하는 것이다.

Hierarchy(1)는 $g_t^{(1)}$를 이용하여 Worker 의 행동 선택에 관여한다. Hierarchy(0)는 U^t와 $\phi_s\left(g_t^{(1)}\right)$를 더한다. 이에 Softmax 함수를 취해 행동에 대한 확률 값을 얻어 이로 행동을 결정한다. Hierarchy(0)이 받은 Hierarchy(1)의 목표는 Hierarchy(2)의 정보까지 포함한 것이다. 다양한 주기 정보를 고려함으로 더욱 정교한 의사 결정이 가능하다. $\phi_{(1)}$은 Hierarchy(1)의 g_t로부터 오는 기울기를 통해서 학습하고, ϕ_s는 Worker 의 행동으로부터 오는 기울기를 통해 학습한다.

Hierarchy(2)는 $A^{(2)}\mathrm{t} = \mathrm{r_t} - \mathrm{V}^{(2)}\mathrm{t}(\mathrm{x_t}, \theta)$ *일 때,* $\nabla g^{(2)}\mathrm{t} = \mathrm{A_t^{(2)}}\nabla\mathrm{dcos}\left(\mathrm{s_{t+20}} - \mathrm{s_t}, \mathrm{g_t^{(2)}}(\theta)\right)$ *를 이용해 학습할 수 있다.* $\mathrm{dcos}\left(\mathrm{s_{t+20}} - \mathrm{s_t}, \mathrm{g_t^{(2)}}(\theta)\right)$ 는 Hierarchy(2)의 $g_t^{(2)}$ 과 상태의 변화가 유사한 정도를 나타낸다. Hierarchy(1)도 같은 방식으로 학습할 수 있다. $A_t^{(1)} = r_t - V_t^{(1)}(x_t, \theta)$ 일 때, $\nabla g^{(1)}\mathrm{t} = \mathrm{A_t^{(1)}}\nabla\mathrm{dcos}\left(\mathrm{s_{t+10}} - \mathrm{s_t}, \mathrm{g_t^{(1)}}(\theta)\right)$를 이용한다.

2.4. experiment

앞서 언급한 Four Rooms 환경에서 실험하였다. 각 모델 당 10M training steps 를 사용하였고, 한 에피소드 당 최대로 취할 수 있는 step 수는 1000 으로 설정하였다. gamma 는 FuN 이 k=0, k=1 각각 0.9, 0.99 를, THE FUN 은 k=0, k=1, k=2 각각 0.9, 0.95, 0.99 의 값을 사용했다. 성능값은 하나의 모델을 5 번 돌려 그 평균 값으로 하였다.

3. Results Experiments

모델간의 성능 비교를 위해 우리는 average reward 와 top 1%reward, 그리고 reward 의 표준편차를 비교하였다.

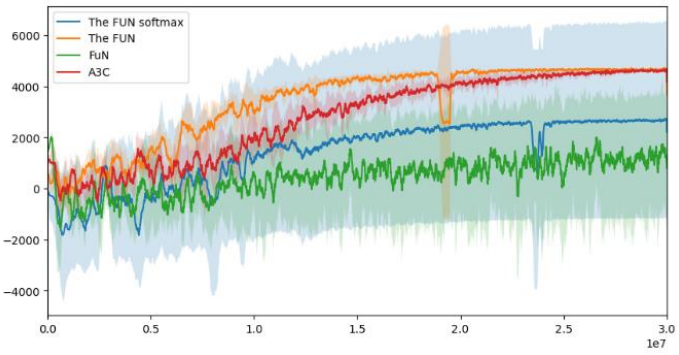


figure2. Training curves(average reward)

가장 먼저 우리는 모델의 평균 성능을 측정하였다. 가장 빨리 학습한 것은 The FUN 모델이며 A3C 는 느리게 학습하지만 최종적으로는 The FUN 모델과 같은 수준의 성능을 보였다. The FUN softmax 모델은 5 번의 실험에서 편차가 크게 나타났고 FuN 모델보다 성능이 살짝 좋은 수준에 불과했다. FuN 모델은 성능의 향상폭이 가장 적었고 제대로 학습되지 않았다.

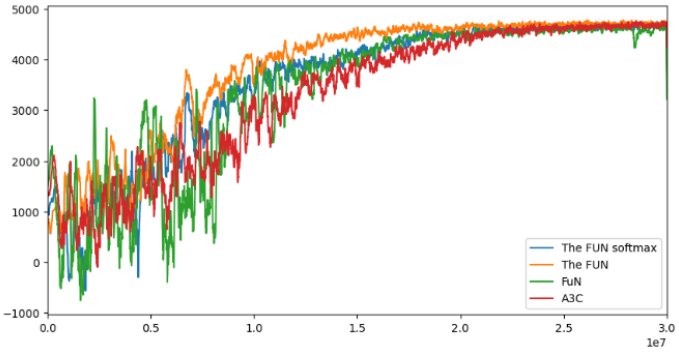


figure3. Training curves(top 1% percentile)

둘째로, 각 모델의 최고 성능을(top 1%) 측정하였다. 모델의 평균 성능과 마찬가지로 The FUN 모델이 가장 좋게 나왔으며, hierarchical 하게 구성된 모든 모델 The FUN softmax, FuN 이 A3C 모델 보다 좋게 나타났다.

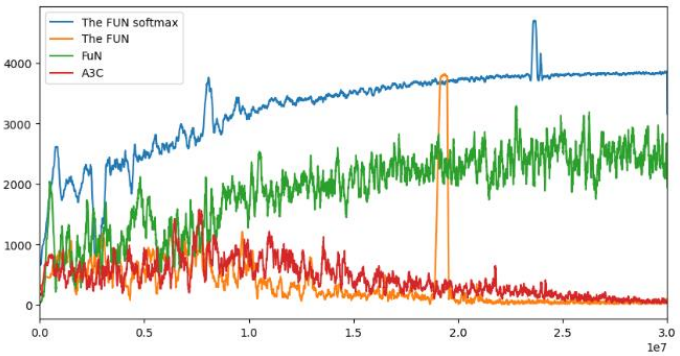


figure4. Training standard deviation

마지막으로 standard deviation 을 측정하였다. The FUN 모델과 A3C 모델 둘이 가장 좋게 나타났다.

4. Discussion

본 연구의 목적은 더 빠르고 안정적으로 학습할 수 있는 다 계층의 HRL 모델을 찾는 것이다.

이를 위해 우리는 서로 다른 방식으로 학습하는 n=2 에이전트 버전을 만들어 비교하였다. 그 결과로 다 계층 HRL 모델 구축에 사용할 수 있는 표준적 학습 방법을 찾았다. 이를 통해 우리는 계층을 추가하는 것이 학습을 더욱 빠르고 안정적으로 만든다는 것을 확인할 수 있었다.

이는 단순히 계층을 많이 쌓는 것이 아닌 적절한 학습 방법이 중요함을 보여준다. 특히, A3C 와 FuN 그리고 THE FUN 모델의 std 비교는 계층을 하나만 쌓는 것은 오히려 학습의 안정성을 크게 떨어뜨린다는 것을 확인할 수 있었다.

추후 연구에서는 수행하려는 taks 의 어떤 요소가 필요 계층 수를 조절하는지 확인할 필요가 있다. 이를 통해서, 계층의 수와 환경 사이의 관계 파악이 가능하고, 각 task 에 맞는 계층 수를 찾아낼 수 있을 것이다.

References

[1] T. D. Kulkarni, K. R. Narasimhan, A. Saeedi, and J. B. T. Bcs, “Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation.”

[2] R. Fox, S. Krishnan, I. Stoica, and K. Goldberg, “Multi-Level Discovery of Deep Options,” Mar. 2017, [Online]. Available: http://arxiv.org/abs/1703.08294

[3] A. S. Vezhnevets *et al.*, “FeUdal Networks for Hierarchical Reinforcement Learning,” Mar. 2017, [Online]. Available: http://arxiv.org/abs/1703.01161

[4] V. Mnih *et al.*, “Asynchronous Methods for Deep Reinforcement Learning,” Feb. 2016, [Online]. Available: http://arxiv.org/abs/1602.01783